

BÁO CÁO PHÂN TÍCH MÃ NGUỒN

DashboardManagerWifiMesh

Dashboard giám sát, WebSocket server và lưu trữ dữ liệu
MSMS

Frontend: React 18, Material UI, ApexCharts

Backend: Node.js, Express, ws

Lưu trữ: SQLite hoặc MongoDB

Giao tiếp: HTTP REST, WebSocket, mDNS

Phiên bản tài liệu: 1.0

Tài liệu được xây dựng từ trạng thái mã nguồn hiện tại

Tháng 07 năm 2026

Mục lục

1	Giới thiệu	1
1.1	Mục tiêu tài liệu	1
2	Kiến trúc tổng thể	1
3	Tổ chức repository và công nghệ	1
4	Khởi động và định tuyến frontend	4
4.1	Các route hiện tại	5
5	Lựa chọn nguồn WebSocket	6
5.1	Thứ tự xác định URL	6
6	WebSocket và HTTP backend	7
6.1	Khởi tạo server	7
6.2	Xử lý một kết nối	8
7	Pipeline nhận và phân tách bản tin	8
7.1	Ba dạng mesh input được chấp nhận	8
8	Schema dữ liệu mesh	9
8.1	Ánh xạ loại cảm biến	10
9	Lưu trữ SQLite	11
9.1	Mô hình dữ liệu dạng long table	11
9.2	Lưu gateway_status	12
10	REST API và dữ liệu lịch sử	12
11	Realtime state ở frontend	13
11.1	MeshRealtimeProvider	13
11.2	Các ngưỡng thời gian	14
11.3	Tính throughput và latency	14
12	Packet loss và server metrics	15

13 SQLite và MongoDB: mức tương thích thực tế	16
14 Khởi chạy và dừng trên Windows	17
14.1 execute.bat	17
14.2 stop.bat	17
15 Những phần giao diện chưa nối hoàn chỉnh	18
16 Rủi ro và đề xuất cải tiến	18
17 Kế hoạch kiểm thử	19
18 Build và triển khai	20
19 Kết luận	21
A Thứ tự đọc mã đề xuất	21

Danh sách hình vẽ

1	Kiến trúc triển khai của DashboardManagerWifiMesh	1
2	Trình tự mount và render frontend	4
3	Thứ tự chọn URL trong wsConfig.js	6
4	Áp dụng nguồn dữ liệu mới không cần sửa cứng mã	7
5	Khởi động backend Node.js	7
6	Vòng đời kết nối WebSocket phía server	8
7	Phân nhánh xử lý message tại backend	8
8	Parser hai lớp của uart_rx	9
9	Chuẩn hóa một packet thành nhiều record SQLite	11
10	Luồng truy vấn lịch sử	13
11	State realtime được chia sẻ cho toàn bộ layout	13
12	Hai luồng số liệu do backend tự sinh	15
13	Cơ chế execute.bat	17

Danh sách bảng

1	Các thư mục và tệp quan trọng	2
2	Thư viện chính	3
3	Bản đồ màn hình	5
4	Ý nghĩa header mesh	10
5	Sensor type và thứ tự giá trị	10
6	Cột bảng Data của SQLite	11
7	REST endpoint	12
8	Ngưỡng và giới hạn realtime	14
9	So sánh adapter hiện tại	16
10	Ma trận kiểm thử đề xuất	19
11	Biến môi trường quan trọng	20

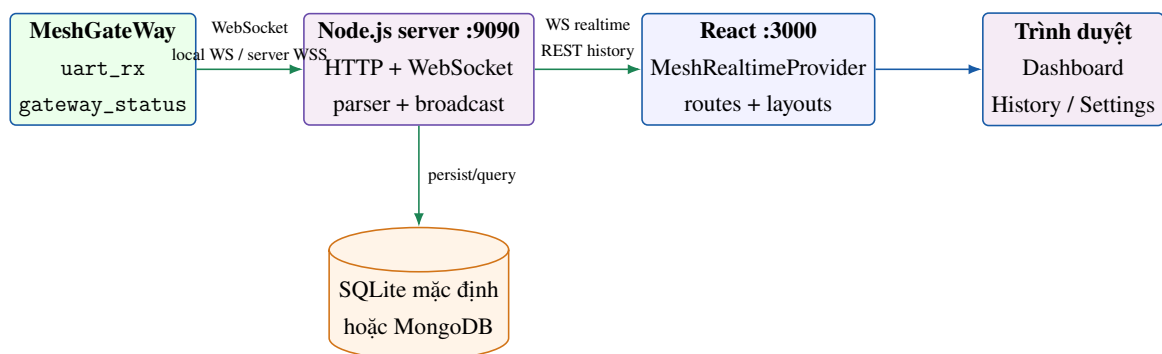
1 Giới thiệu

DashboardManagerWifiMesh là ứng dụng giám sát gồm frontend React và backend Node.js chạy trong cùng repository. Backend nhận bản tin từ MeshGateWay, phân tách payload compact của mạng mesh, lưu từng giá trị đo vào cơ sở dữ liệu, rồi phát bản tin cho các trình duyệt đang mở dashboard. Frontend kết hợp snapshot HTTP với stream WebSocket để dựng trạng thái gateway, node, cảm biến, lịch sử và hiệu năng server.

1.1 Mục tiêu tài liệu

- giải thích đầy đủ đường đi của dữ liệu từ gateway tới từng màn hình;
- mô tả parser hai lớp `uart_rx` và schema compact của firmware;
- phân biệt dữ liệu realtime trong RAM với dữ liệu lịch sử trong database;
- làm rõ cách chọn local/server/custom WebSocket;
- chỉ ra chức năng đã hoạt động, chức năng giao diện mẫu và các điểm không tương thích cần sửa.

2 Kiến trúc tổng thể



Hình 1: Kiến trúc triển khai của DashboardManagerWifiMesh

Trong chế độ development, `npm run dev` dùng `concurrently` để chạy React và WebSocket server. Hai tiến trình không phải một server duy nhất: React phục vụ trang tại cổng 3000; Node.js lắng nghe HTTP và WebSocket tại cổng 9090.

3 Tổ chức repository và công nghệ

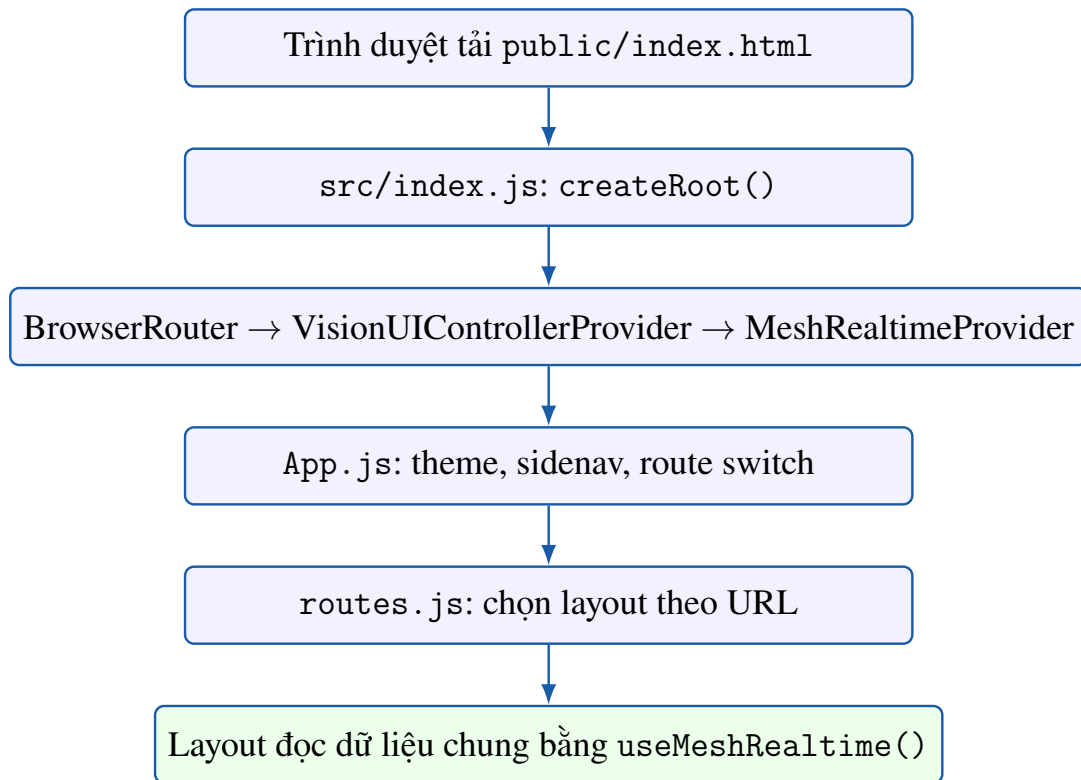
Bảng 1: Các thư mục và tệp quan trọng

Đường dẫn	Trách nhiệm
public/index.html	Vỏ HTML và điểm mount root.
src/index.js	Bọc Router, UI context, realtime context và App.
src/App.js	Theme, sidenav và runtime routing.
src/routes.js	Bảng route đồng thời là nguồn tạo menu.
src/context/ meshRealtime.js	Kết nối WS dùng chung, parser, registry và state realtime.
src/utils/wsConfig.js	Chọn URL local/server/custom và lưu localStorage.
src/utils/ meshUdpJsonSchema.js	Chuẩn hóa payload compact, sensor và đơn vị.
src/layouts/	Các trang Dashboard, Gateway, Nodes, Sensors, History, Alerts, OTA và Settings.
websocket-server/ server.js	HTTP REST, WS, mDNS, heartbeat, broadcast và server metrics.
websocket-server/ sqlite-store.js	SQLite WAL, ghi dữ liệu, history và CSV.
websocket-server/ mongodb-store.js	Adapter MongoDB cũ, chưa đồng nhất interface mới.
execute.bat/stop.bat	Khởi chạy và dừng cây tiến trình trên Windows.

Bảng 2: Thư viện chính

Thư viện	Vai trò
React 18, React Router 5	Cây component, state và điều hướng SPA.
Material UI, Emotion	Component UI, theme, CSS-in-JS và RTL.
ApexCharts	Biểu đồ realtime và history.
Express, CORS	REST API và middleware HTTP.
ws	WebSocket server, client registry, ping/pong.
sqlite + sqlite3	CSDL file local, chế độ WAL.
mongoose	Schema và truy vấn MongoDB.
systeminformation	CPU, RAM, nhiệt độ và uptime máy chạy backend.
bonjour-service	Quảng bá systemmsems.local bằng mDNS.
Swagger	Tài liệu REST tại /api-docs.

4 Khởi động và định tuyến frontend



Hình 2: Trình tự mount và render frontend

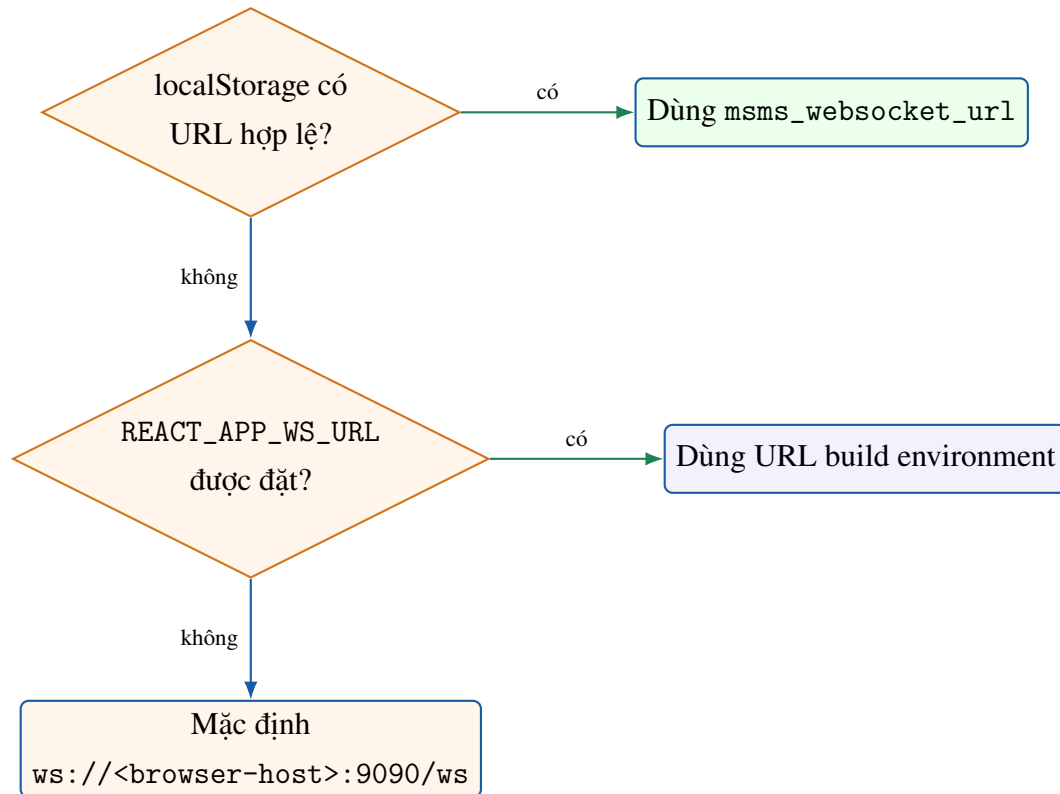
4.1 Các route hiện tại

Bảng 3: Bản đồ màn hình

Route	Layout	Mục đích
/dashboard	Dashboard	Tổng quan root, node, throughput và cảm biến mới nhất.
/gateway	Gateway	CPU/RAM/RSSI/pin/uptime của MeshGateWay.
/nodes	Nodes	Danh sách registry node và trạng thái online.
/nodes/:nodeId	NodeDetail	Chi tiết một node; route ẩn khỏi menu.
/sensors	Sensors	Các giá trị sensor realtime.
/history	History	Chọn MAC/sensor/field, biểu đồ và export CSV.
/alerts	Alerts	Hiển thị lỗi runtime và điều kiện cảnh báo.
/ota/overview	OtaOverview	Tổng quan OTA.
/ota/firmware	FirmwareManagement	Giao diện quản lý firmware.
/ota/fota	UpdateNodes	Giao diện FOTA.
/settings/system	SystemSettings	Chọn nguồn WS và các cấu hình UI.
/about	About	Thông tin hệ thống.

5 Lựa chọn nguồn WebSocket

5.1 Thứ tự xác định URL

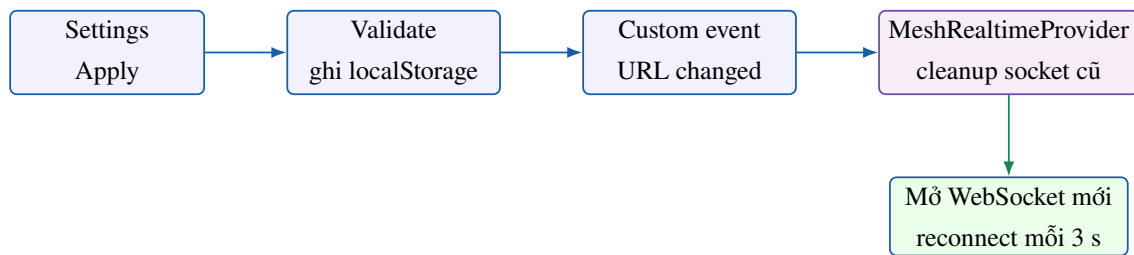


Hình 3: Thứ tự chọn URL trong wsConfig.js

Ba lựa chọn trên giao diện Settings:

- **Local server:** `ws://<window.location.hostname>:9090/ws`. Khi mở frontend bằng `localhost:3000`, URL là `ws://localhost:9090/ws`.
- **Internet server:** `wss://systemmsems.msems.click/ws`.
- **Custom URL:** người dùng nhập URL bắt đầu bằng `ws://` hoặc `wss://`.

`saveWebSocketSourceSettings()` kiểm tra URL, ghi source và URL vào `localStorage`, sau đó phát custom event `msms:websocket-config-changed`. Provider nhận event, đổi dependency url, đóng socket cũ và mở socket mới. Khác với URL trên firmware gateway, cấu hình frontend này bền qua refresh trình duyệt nhưng chỉ thuộc profile trình duyệt hiện tại.

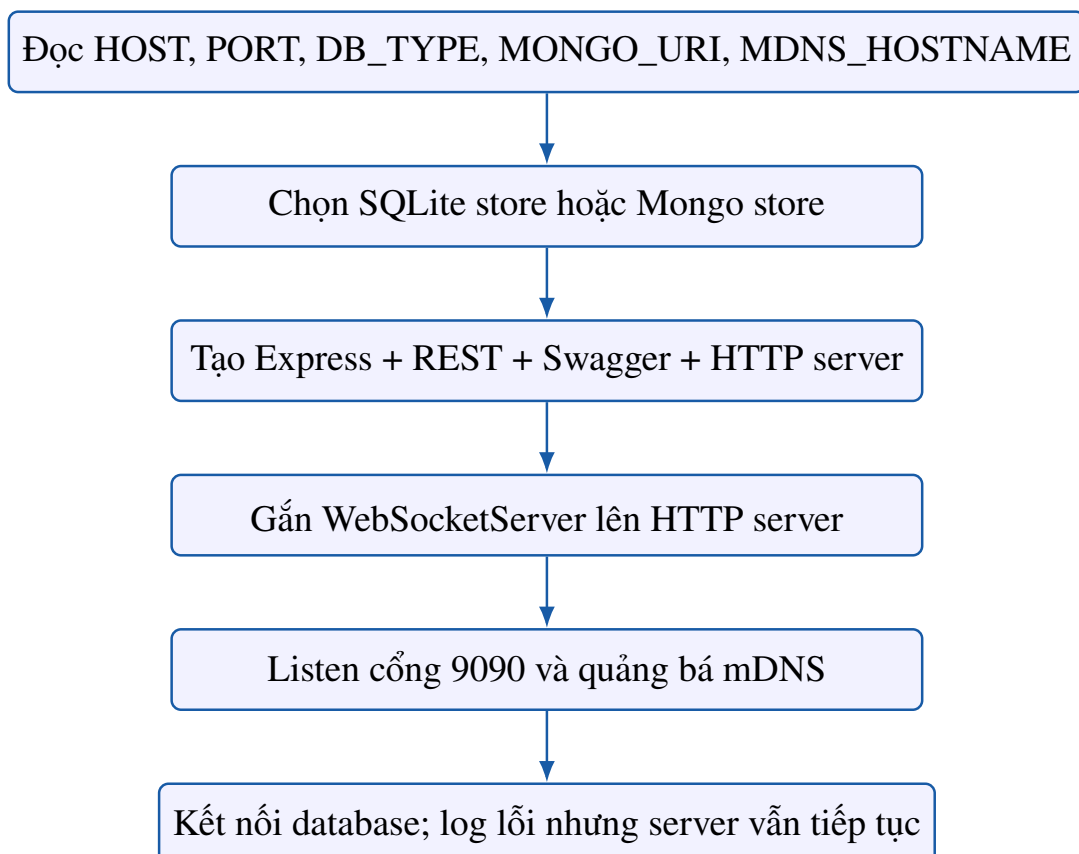


Hình 4: Áp dụng nguồn dữ liệu mới không cần sửa cứng mã

6 WebSocket và HTTP backend

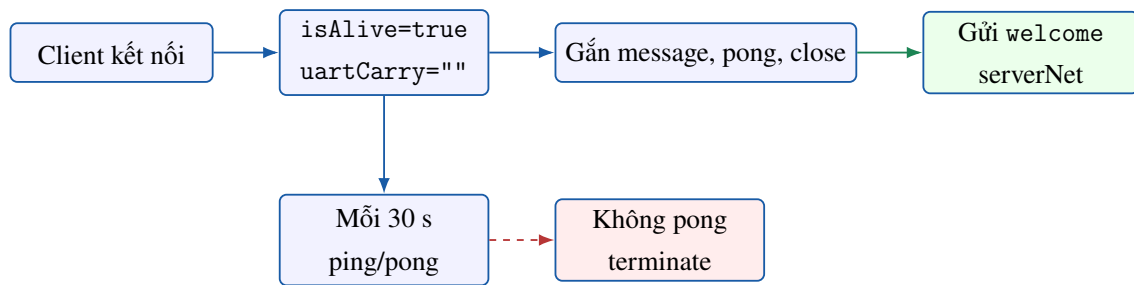
6.1 Khởi tạo server

Backend dùng một `http.Server` chung cho Express và `WebSocketServer`. Mặc định bind `0.0.0.0:9090`. mDNS quảng bá `systemmsems.local`, service HTTP với TXT `service=websocket,path=/ws`. WebSocket server không giới hạn path trong constructor, vì vậy kết nối tới root hoặc `/ws` đều đi vào cùng server.



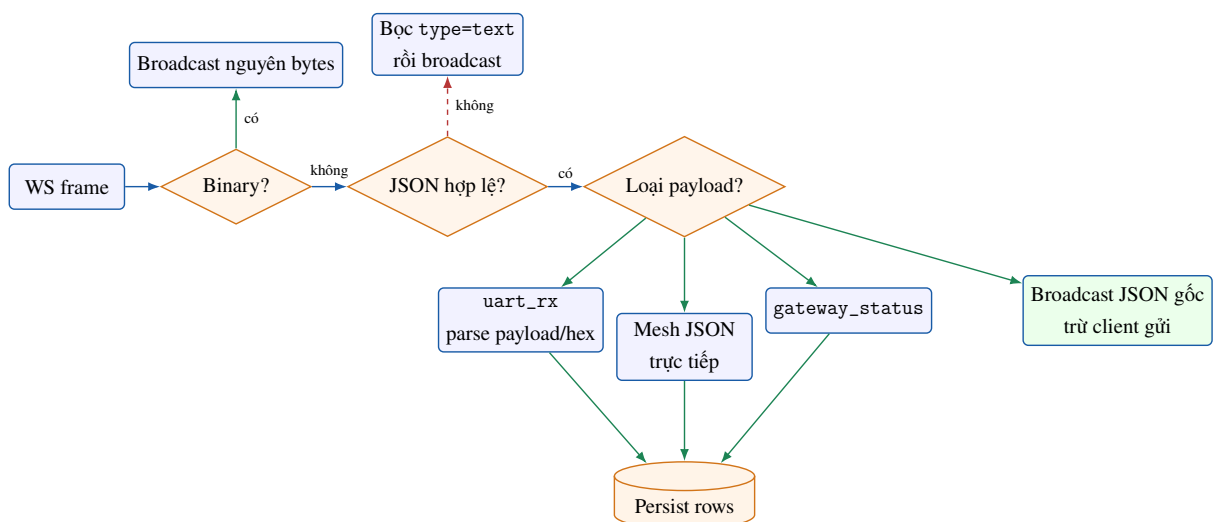
Hình 5: Khởi động backend Node.js

6.2 Xử lý một kết nối



Hình 6: Vòng đời kết nối WebSocket phía server

7 Pipeline nhận và phân tách bản tin

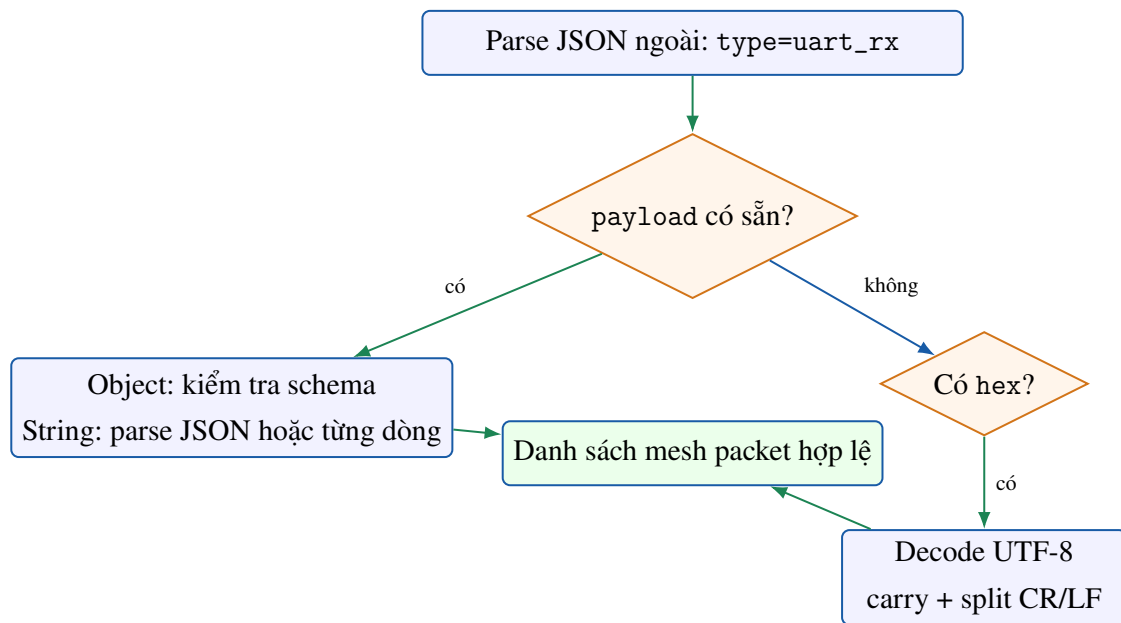


Hình 7: Phân nhánh xử lý message tại backend

Lỗi parser hoặc database được bắt riêng. Một JSON hợp lệ vẫn được broadcast với cấu trúc ban đầu ngay cả khi persist thất bại. Đây là quyết định tốt cho realtime: UI không bị biến bản tin hợp lệ thành text chỉ vì database gặp lỗi.

7.1 Ba dạng mesh input được chấp nhận

1. JSON mesh trực tiếp chứa v, n, M hoặc i, và mảng p.
2. `uart_rx.payload` là object hoặc chuỗi JSON.
3. `uart_rx.hex` là UTF-8 mã hex; server ghép phần dòng chưa hoàn chỉnh trong `state.uartCarry`, giới hạn phần carry 4096 ký tự.



Hình 8: Parser hai lớp của uart_rx

8 Schema dữ liệu mesh

Listing 1: Payload compact tiêu biểu

```

{
  "v": 1,
  "seq": 88,
  "n": 2,
  "M": "B4:BF:E9:34:0C:CC",
  "t": "2012-01-01T00:02:02",
  "ver": "0.0.1",
  "err": ["0x51"],
  "p": [[1, 6, 27.594, 42.450]]
}
    
```

Bảng 4: Ý nghĩa header mesh

Trường	Kiểu	Ý nghĩa
v	number	Phiên bản schema. Parser frontend khai báo hằng số 2 nhưng không bắt buộc bằng đúng 2.
seq	number	Số thứ tự dùng tính packet loss tại backend.
n	number	Tầng mesh.
M / i	string	Định danh node; mã hiện tại ưu tiên M.
t	ISO string	RTC của node, dùng ước lượng latency.
ver	string	Phiên bản firmware.
err	string array	Mã lỗi dạng 0xNN.
p	array	Các hàng [port, type, v0, ...].

8.1 Ánh xạ loại cảm biến

Bảng 5: Sensor type và thứ tự giá trị

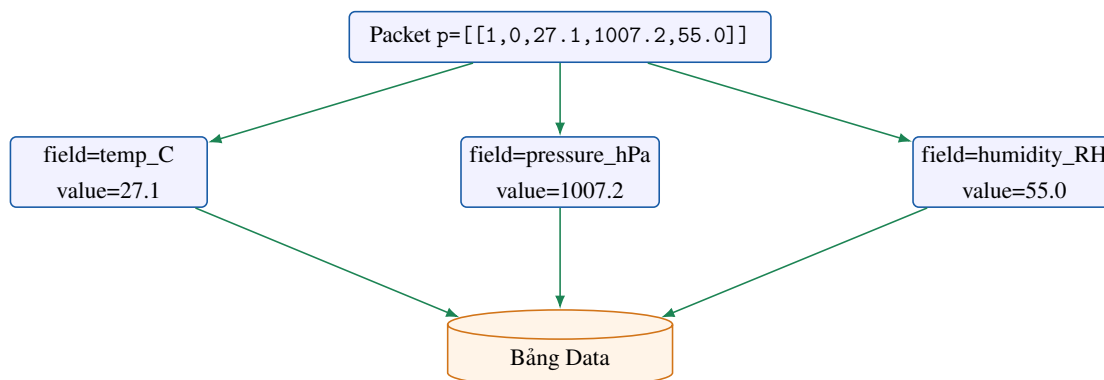
ID	Sensor	Các trường theo thứ tự
0	BME280	temp_C, pressure_hPa, humidity_RH
1	MH-Z14A	co2_ppm, temp_C
2	PMS7003	pm1_0_ugm3, pm2_5_ugm3, pm10_ugm3
3	DHT22	temp_C, humidity_RH
4	AHT10	temp_C, humidity_RH
5	DHT11	temp_C, humidity_RH
6	HTU21D	temp_C, humidity_RH

Frontend giới hạn tối đa năm giá trị trên một row. wirePort hợp lệ từ 1 đến 255 và portIndex = wirePort - 1. Mã isSensorTypeWireValue() hiện kiểm tra tối đa SENSOR_AHT10 (ID 4), trong khi registry định nghĩa cả DHT11 và HTU21D; đây là điểm không đồng nhất cần sửa nếu hàm kiểm tra này được dùng để reject type.

9 Lưu trữ SQLite

9.1 Mô hình dữ liệu dạng long table

Mỗi giá trị sensor trở thành một dòng. Ví dụ một row BME280 có ba giá trị sẽ tạo ba record cùng MAC, thời gian, port và sensorName nhưng khác field.



Hình 9: Chuẩn hóa một packet thành nhiều record SQLite

Bảng 6: Cột bảng Data của SQLite

Cột	Kiểu	Nội dung
id	INTEGER	Khóa chính tự tăng.
mac	TEXT	M hoặc i.
deviceTime	TEXT	Chuỗi thời gian thiết bị.
serverTime	TEXT	ISO time tại lúc backend nhận.
meshLevel	INTEGER	Tầng mesh.
schemaVersion	INTEGER	Trường v.
packetLoss	REAL	Tỷ lệ mất gói nếu có.
port	INTEGER	Port trên wire.
sensorType	INTEGER	ID sensor.
sensorName	TEXT	Tên đã ánh xạ.
field	TEXT	Tên đại lượng.
value	REAL	Giá trị số.
nodeTimeValid	INTEGER	Hiện được ghi 0 trong adapter SQLite.

Cột	Kiểu	Nội dung
sourceIp	TEXT	IP socket gửi tới backend.

SQLite bật PRAGMA journal_mode=WAL, tạo index theo serverTime DESC và index ghép (mac,field,serverTime). Chế độ execute.bat đặt DB_TYPE=sqlite; database mặc định là websocket-server/mesh-data.sqlite, kèm các file WAL/SHM khi đang chạy.

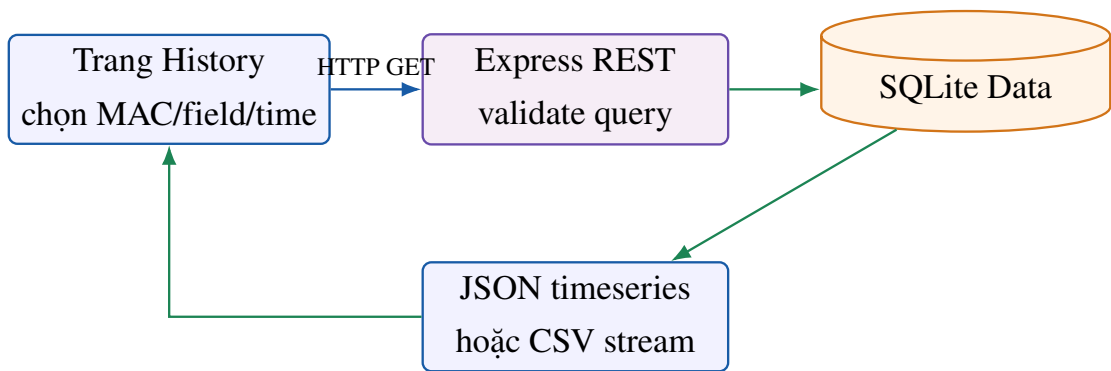
9.2 Lưu gateway_status

Adapter SQLite chuyển năm metrics thành record có sensorName=Gateway: CPU load, RAM used, RSSI, battery voltage và uptime. MAC gateway được lấy lần lượt từ mac, sta_mac, sta_ip, source IP hoặc chuỗi gateway.

10 REST API và dữ liệu lịch sử

Bảng 7: REST endpoint

Endpoint	Kết quả
GET /api/nodes	Snapshot node theo record mới nhất.
GET /api/history/meta?limit=	Danh sách MAC, lastSeen và sensor names.
GET /api/history/series	Chuỗi tối đa 5000 điểm theo MAC, sensor, field và thời gian.
GET /api/history/export	Stream CSV toàn bộ history của một MAC theo khoảng thời gian.
GET /api-docs	Swagger UI.

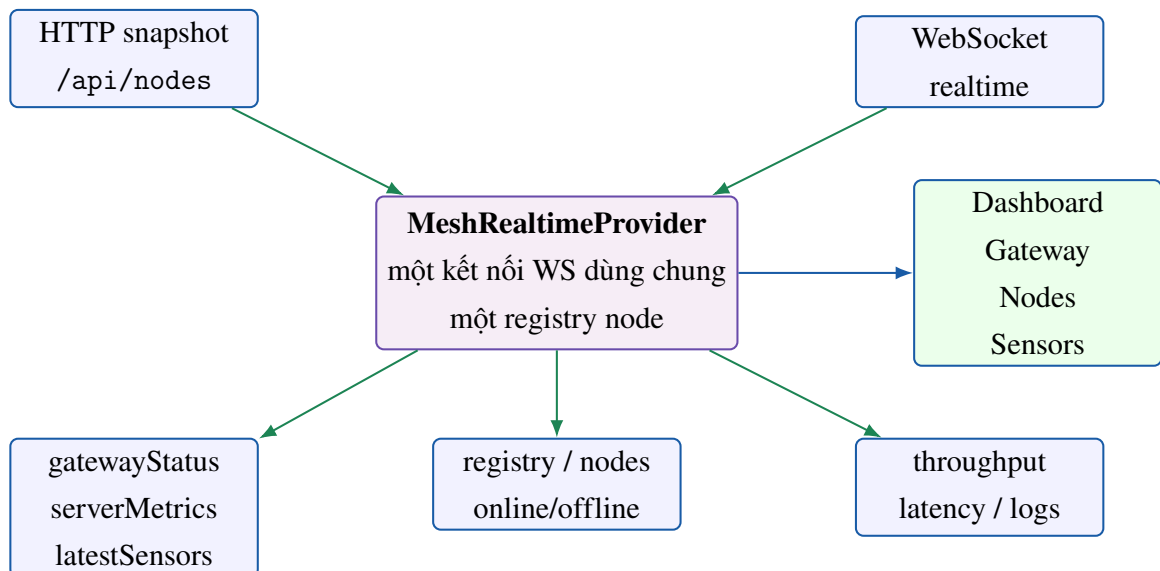


Hình 10: Luồng truy vấn lịch sử

Export SQLite đọc theo chunk 5000 dòng và ghi trực tiếp vào HTTP response, tránh nạp toàn bộ file CSV vào RAM. Tuy nhiên nội dung CSV được ghép bằng template string mà chưa escape dấu phẩy, dấu ngoặc kép hay xuống dòng; tên sensor hiện tại an toàn nhưng nên dùng hàm CSV escaping chuẩn.

11 Realtime state ở frontend

11.1 MeshRealtimeProvider



Hình 11: State realtime được chia sẻ cho toàn bộ layout

Provider fetch snapshot một lần khi URL thay đổi, sau đó mở WebSocket. Khi socket đóng, nó thử lại sau 3 giây. Cleanup hủy timer, tháo handler và đóng socket, tránh tạo nhiều kết nối khi người dùng đổi nguồn.

11.2 Các ngưỡng thời gian

Bảng 8: Ngưỡng và giới hạn realtime

Hằng số	Giá trị	Ý nghĩa
DEVICE_STALE_MS	15 s	Không có device message thì root status chuyển waiting/offline.
NODE_STALE_MS	20 s	Dùng thống kê online/offline.
STALE_MS_NODES	theo context	Dùng field online trong danh sách node.
THROUGHPUT_WINDOW_SECONDS	60 s	Cửa sổ đồ thị bytes/kbps.
MAX_DEBUG_LOGS	120	Giới hạn log trong RAM.
Reconnect	3 s	Thời gian mở lại frontend WS.
Server heartbeat	30 s	Ping/pong và loại zombie socket.

Có nhiều hằng stale khác nhau giữa context và hook cũ useMeshNodesFromWebSocket (7 giây). Hiện các hook ưu tiên trả dữ liệu từ context nếu provider tồn tại, nhưng code fallback còn giữ logic trùng lặp. Nên gom các timeout về một module cấu hình để tránh cùng node nhưng màn hình khác nhau hiển thị trạng thái khác nhau.

11.3 Tính throughput và latency

Mỗi message text được đếm số byte UTF-8 theo giây. Cứ mỗi giây, provider lấy bucket của giây trước, giữ 60 điểm và tính:

$$\text{kbps} = \frac{\text{bytes} \times 8}{1000}.$$

Latency được tính từ thời gian browser nhận trừ RTC trong payload. Giá trị bị ép tối thiểu 0. Nếu node dùng thời gian mặc định năm 2012 hoặc RTC chưa đồng bộ, latency rất lớn và không phản ánh trễ mạng; UI cần đánh dấu time invalid thay vì coi đó là network latency.

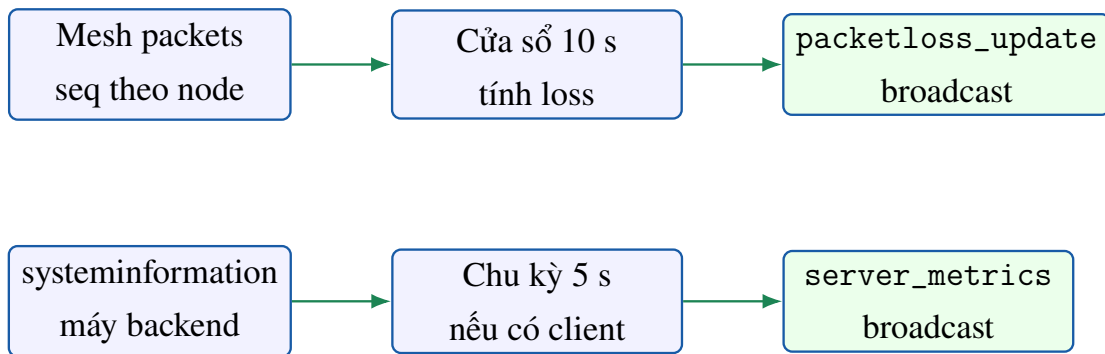
12 Packet loss và server metrics

Backend giữ seqStats theo định danh node. Mỗi 10 giây:

$$expected = maxSeq - minSeq + 1, \quad loss = \frac{expected - count}{expected} \times 100\%.$$

Kết quả được broadcast bằng packetloss_update và thử persist một packet hệ thống. Cách tính này giả định seq tăng đều, không xử lý trùng gói và reset/wrap sequence đầy đủ; ngưỡng $expected > 10000$ chỉ là bảo vệ sơ bộ.

Mỗi 5 giây, nếu có ít nhất một WebSocket client, backend dùng systeminformation tạo server_metrics: CPU load, RAM, nhiệt độ CPU, uptime và received time. Đây là metrics của **máy chạy Node.js**, không phải MeshGateWay. Frontend giữ series riêng cho server và gateway.



Hình 12: Hai luồng số liệu do backend tự sinh

13 SQLite và MongoDB: mức tương thích thực tế

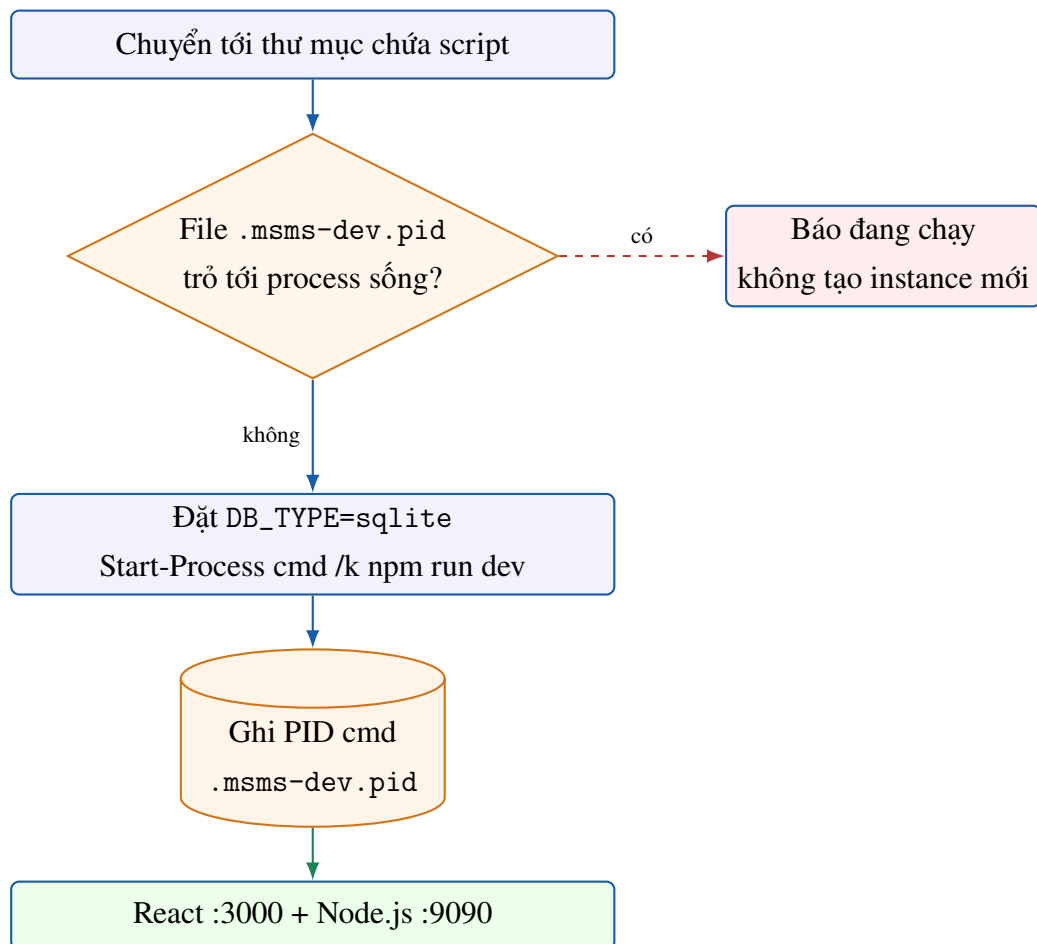
Bảng 9: So sánh adapter hiện tại

Khả năng	SQLite	MongoDB
Định danh M	Có, ưu tiên M	Không; validator và builder chỉ dùng i
Persist mesh	Có	Có nếu payload có i
Persist gateway status	Có	Không export hàm tương ứng
Node snapshot	mac	ip
History metadata	getHistoryMacs	getHistoryIps
History series input	mac,sensorName,field	ip,field
CSV export	Có	Không
Phù hợp server.js hiện tại	Có	Chưa tương thích đầy đủ

Kết luận: mặc dù DB_TYPE mặc định trong server.js là mongodb, REST handler hiện gọi interface của SQLite. Chạy bằng execute.bat an toàn hơn vì script đặt DB_TYPE=sqlite. Muốn dùng MongoDB phải cập nhật adapter theo interface thống nhất hoặc tạo lớp contract chung và test cho cả hai backend.

14 Khởi chạy và dừng trên Windows

14.1 execute.bat



Hình 13: Cơ chế execute.bat

14.2 stop.bat

stop.bat yêu cầu quyền Administrator, đọc PID đã lưu và chạy `taskkill /PID <pid> /T /F`. Sau đó script quét local endpoint cổng 3000 và 9090 bằng `netstat` để dọn process mồ côi. Việc dùng `/T` là quan trọng vì PID file là PID của `cmd.exe`, còn React và Node là tiến trình con.

Listing 2: Cách sử dụng

```

cd /d "D:\Engineering Project\DashboardManagerWifiMesh"
execute.bat

rem Stop the program:
stop.bat
  
```

15 Những phần giao diện chưa nổi hoàn chỉnh

Không phải mọi nút trong UI đều có backend tương ứng:

- các nút Save changes, Reset to default của system setting và nhiều System Actions hiện chủ yếu là giao diện, chưa gọi API;
- thông tin System Uptime, Firmware/Hardware Version, Last Restart và Memory Usage trong card System Information có giá trị mẫu viết cứng;
- các trang OTA/FOTA có giao diện nhưng firmware gateway hiện chưa nổi pipeline OTA hoàn chỉnh;
- notification email và các nút restart/factory reset chưa có endpoint điều khiển thực tế trong server.js.

Khi viết tài liệu vận hành cần phân biệt “màn hình tồn tại” với “hành động đã được backend và firmware thực thi”.

16 Rủi ro và đề xuất cải tiến

1. **Chuẩn hóa Store interface:** định nghĩa contract chung connect, persistMeshPacket, persistGatewayStatus, getNodeSnapshot, getHistoryMacs, getHistorySeries và export CSV.
2. **Thống nhất định danh M:** đổi MongoDB từ field ip sang mac hoặc hỗ trợ migration/alias rõ ràng.
3. **Giảm parser trùng lặp:** parser UART/mesh đang có ở server, context và hai hook. Tách shared package hoặc tạo bộ fixture contract.
4. **Bảo vệ WebSocket và REST:** hiện chưa thấy authentication, authorization, rate limit hay giới hạn kích thước message ở lớp ứng dụng.
5. **Giới hạn log:** server log toàn bộ text nhận được; telemetry dày có thể làm terminal chậm và lộ dữ liệu. Nên có log level và truncate.
6. **Database unavailable:** server tiếp tục chạy sau lỗi connect, nhưng các lệnh store tiếp theo có thể liên tục lỗi. Nên expose health endpoint và trạng thái degraded.
7. **CSV escaping:** dùng thư viện CSV hoặc escape đúng RFC.
8. **RTC validity:** nhận biết năm mặc định/RTC chưa sync trước khi tính latency và lưu device time.
9. **Packet loss:** xử lý duplicate, reset và wrap sequence bằng sliding window theo node.
10. **CORS và network exposure:** backend bind 0.0.0.0 và CORS mở; cần cấu hình origin allowlist khi triển khai ngoài mạng lab.

17 Kế hoạch kiểm thử

Bảng 10: Ma trận kiểm thử đề xuất

Nhóm	Cá kiểm thử	Kết quả mong đợi
Parser	Mesh trực tiếp có M	Parse node, sensor và persist SQLite.
Parser	uart_rx.payload string	Parse đúng JSON lớp trong.
Parser	Hex bị chia nhiều frame	Carry ghép đúng một dòng.
Parser	Row có string/NaN/type lạ	Trả lỗi có kiểm soát, server không chết.
Realtime	Ngắt backend	UI chuyển wsOpen false, reconnect 3 s.
Realtime	Node im lặng 20 s	Thống kê offline cập nhật.
Config	Local/server/custom	Socket cũ đóng, socket mới mở; refresh vẫn giữ URL.
SQLite	Nhiều writer/read history	WAL cho phép truy vấn khi đang ghi.
History	from/to/limit	Thứ tự tăng dần và giới hạn đúng.
CSV	Hàng dữ liệu lớn	Stream theo chunk, file không thiếu dòng.
WebSocket	Client không pong	Server terminate sau heartbeat.
Windows	execute hai lần	Lần hai báo PID đang chạy.
Windows	stop sau PID mở còi	Dọn process theo cổng 3000/9090.

Nhóm	Ca kiểm thử	Kết quả mong đợi
MongoDB	Chạy toàn bộ API	Hiện dự kiến thất bại; dùng test để dẫn dắt sửa adapter.

18 Build và triển khai

Listing 3: Cài đặt và chạy development

```
cd DashboardManagerWifiMesh
npm install
npm install --prefix websocket-server
npm run dev
```

Listing 4: Chạy riêng và build frontend

```
npm run start:react
npm run ws-server
npm run build
```

Bảng 11: Biến môi trường quan trọng

Biến	Mặc định	Ý nghĩa
WS_PORT / PORT	9090	Cổng HTTP + WebSocket.
WS_HOST	0.0.0.0	Interface bind.
DB_TYPE	mongodb trong server	Adapter database; execute.bat ghi đè sqlite.
MONGO_URI	local MeshData	URI MongoDB.
SQLITE_DB_PATH	mesh-data.sqlite	File SQLite.
MDNS_HOSTNAME	systemmsems	Hostname mDNS.
REACT_APP_WS_URL	local URL	URL frontend được đóng vào build.

19 Kết luận

Dự án đã có kiến trúc realtime hợp lý: một provider WebSocket dùng chung, backend broadcast bản tin gốc, SQLite chuẩn hóa dữ liệu theo field và REST API phục vụ history. Local mode hiện chạy cổng 9090, lưu SQLite và quảng bá `systemmsems.local`. Những việc ưu tiên trước khi triển khai ổn định là chuẩn hóa adapter MongoDB, loại bỏ parser trùng lặp, bảo vệ API/WS, phân biệt chức năng UI mẫu với hành động thật và bổ sung test contract cho schema M.

A Thứ tự đọc mã đề xuất

Tệp	Nội dung nên đọc
<code>package.json</code>	Script chạy đồng thời frontend/backend.
<code>src/index.js</code>	Cây provider.
<code>src/context/meshRealtime.js</code>	Luồng realtime trung tâm.
<code>src/utils/wsConfig.js</code>	Local/server/custom URL.
<code>src/utils/meshUdpJsonSchema.js</code>	Schema và sensor mapping.
<code>src/routes.js</code>	Route và menu.
<code>src/layouts/system-settings/index.js</code>	Giao diện đổi nguồn dữ liệu.
<code>websocket-server/server.js</code>	Toàn bộ HTTP/WS pipeline.
<code>websocket-server/sqlite-store.js</code>	Database mặc định thực tế.
<code>websocket-server/mongodb-store.js</code>	Adapter cũ cần đồng bộ.
<code>execute.bat</code>	Khởi chạy và PID file.
<code>stop.bat</code>	Dừng cây process và dọn cổng.